



MMI CondorNet™ HAL-SSN NeuroNetics™

Research Grade Addendum - Mathematical Formulations &
Implementations

ADDENDUM TO INITIAL QUALITATIVE RESEARCH PAPER ENTITLED:

Machines Fixing Machines Real-Time Diagnostic and Neurosurgical Repair in Neural Networks.pdf

AUTHOR: DR. T. JERRY MAHABUB, PH.D.

FEBRUARY 12, 2026

“WE’RE NOT JUST TRAINING A NEURAL NET BRAIN, WE’RE BUILDING A NEURAL NET Machine–Machine Interface Organism — a self-observing, self-stabilizing, self-healing cognitive system equipped with the instrumentation to detect its own pathologies and perform autonomous neurosurgery on its internal operators.”

Table of Contents:

Gap 1: Lyapunov Stability Certificates for Neural CDEs

1.1 Mathematical Foundation

- 1.1.1 Lyapunov Function Construction for ETD-1
- 1.1.2 Continuous Algebraic Riccati Equation (CARE)
- 1.1.3 Exponential Stability Proof
- 1.1.4 Region of Attraction Computation
- 1.2 Stability-Preserving Repair Constraints
- 1.2.1 Negative Definiteness Verification
- 1.2.2 Spectral Bounds from Lyapunov Matrix
- 1.3 Implementation
- 1.3.1 LyapunovStabilityMonitor Class
- 1.3.2 CARE Solver Integration
- 1.3.3 Real-time Monitoring Loop
- 1.4 CondorNet Integration
- 1.4.1 ETD-1 Master Equation Connection
- 1.4.2 4-Block State Lyapunov Analysis

Gap 2: Gradient Pathology Detection Beyond Spectral Radius

2.1 Mathematical Framework

- 2.1.1 Gradient Norm Metrics
- 2.1.2 Layer-wise Gradient Ratios
- 2.1.3 Hessian Conditioning
- 2.1.4 Gradient Variance Analysis
- 2.2 Pathology Classification
- 2.2.1 Vanishing Gradient Detection
- 2.2.2 Exploding Gradient Detection
- 2.2.3 Oscillating Gradient Detection
- 2.3 Implementation
- 2.3.1 GradientPathologyMonitor Class
- 2.3.2 Layer-wise Analysis
- 2.3.3 Hessian Approximation via Power Iteration
- 2.4 Repair Strategies
- 2.4.1 Learning Rate Adjustment
- 2.4.2 Gradient Clipping
- 2.4.3 Optimizer Switching

Gap 3: Information-Theoretic Health Metrics

3.1 Mathematical Foundation

- 3.1.1 Shannon Entropy
- 3.1.2 Mutual Information
- 3.1.3 KL Divergence
- 3.1.4 Fisher Information Matrix
- 3.2 Neural Network Health Indicators
- 3.2.1 Activation Entropy (Mode Collapse Detection)
- 3.2.2 Inter-block Mutual Information

- 3.2.3 Distribution Drift via KL Divergence 3.3 Implementation
- 3.3.1 InformationTheoreticMonitor Class
- 3.3.2 Histogram-based Entropy Estimation
- 3.3.3 Gaussian Copula MI Estimation 3.4 CondorNet 4-Block Analysis
- 3.4.1 $I(h;v)$, $I(h;m)$, $I(h;r)$ Computation
- 3.4.2 Regime Shift Detection

Gap 4: Control-Theoretic Repair Strategies

4.1 Mathematical Framework

- 4.1.1 Model Predictive Control (MPC) Formulation
- 4.1.2 Adaptive Control Laws
- 4.1.3 Optimal Control via Hamilton-Jacobi-Bellman 4.2 MPC for Neural Network Repair
- 4.2.1 Cost Function Design
- 4.2.2 Constraint Formulation (Spectral, Lyapunov)
- 4.2.3 Receding Horizon Implementation 4.3 Implementation
- 4.3.1 MPCRepairController Class
- 4.3.2 SciPy Optimization Integration
- 4.3.3 Constraint Violation Handling 4.4 Adaptive Control Implementation
- 4.4.1 Lyapunov-based Adaptation
- 4.4.2 Parameter Update Laws

Gap 5: MTBF/MTTR Reliability Modeling

5.1 Mathematical Foundation

- 5.1.1 Failure Rate $\lambda(t)$
- 5.1.2 Reliability Function $R(t)$
- 5.1.3 $MTBF = \int R(t)dt$
- 5.1.4 Availability $A = MTBF/(MTBF+MTTR)$ 5.2 Failure Mode Classification
- 5.2.1 Spectral Instability
- 5.2.2 Gradient Pathologies
- 5.2.3 Logic Drift
- 5.2.4 Distribution Shift 5.3 Implementation
- 5.3.1 ReliabilityMonitor Class
- 5.3.2 Failure Event Logging
- 5.3.3 Repair Duration Tracking 5.4 Availability Calculation
- 5.4.1 Real-time Metrics
- 5.4.2 SLA Compliance Checking

Gap 6: Causal Inference for Fault Localization (NOVEL)

6.1 Mathematical Foundation

- 6.1.1 Structural Causal Models (SCM)

- 6.1.2 Do-Calculus and Interventions
- 6.1.3 Average Causal Effect (ACE)
- 6.1.4 Counterfactual Reasoning 6.2 Causal Graph Learning
- 6.2.1 PC Algorithm
- 6.2.2 Constraint-based Discovery 6.3 Implementation
- 6.3.1 CausalFaultLocalizer Class
- 6.3.2 DoWhy Integration
- 6.3.3 Causal Effect Estimation 6.4 Beyond Correlation
- 6.4.1 Ochiai/Tarantula Baseline
- 6.4.2 Causal vs Correlation Ranking

Gap 7: Runtime Monitoring Overhead Analysis

7.1 Mathematical Framework

- 7.1.1 Overhead Ratio $\eta = C_{\text{total}}/C_{\text{base}}$
- 7.1.2 Pareto Frontier (Accuracy vs Speed)
- 7.1.3 Adaptive Monitoring Cost Model 7.2 Profiling Methodology
- 7.2.1 Baseline Inference Timing
- 7.2.2 Per-metric Cost Measurement 7.3 Implementation
- 7.3.1 AdaptiveMonitor Class
- 7.3.2 Mode Switching Logic
- 7.3.3 Overhead Budget Management 7.4 Optimization Strategies
- 7.4.1 Sparse Monitoring
- 7.4.2 Amortized Computation

Gap 8: Adversarial Robustness in Repair Mechanisms

8.1 Mathematical Framework

- 8.1.1 Adversarial Perturbation: $x_{\text{adv}} = x + \delta$
- 8.1.2 Certified Robustness Radius
- 8.1.3 Lipschitz Constraints 8.2 Attack Models
- 8.2.1 FGSM (Fast Gradient Sign Method)
- 8.2.2 PGD (Projected Gradient Descent)
- 8.2.3 Poisoning Attacks 8.3 Implementation
- 8.3.1 AdversarialRobustnessTester Class
- 8.3.2 Attack Generation
- 8.3.3 Robustness Verification 8.4 Defense Mechanisms
- 8.4.1 Adversarial Training
- 8.4.2 Input Validation

Gap 9: Self-Healing Distributed Systems

9.1 Mathematical Framework

- 9.1.1 Byzantine Fault Tolerance: $n \geq 3f+1$
- 9.1.2 Consensus Protocols (Raft, Paxos)
- 9.1.3 CAP Theorem Tradeoffs 9.2 Distributed Architecture
- 9.2.1 Multi-node Coordination
- 9.2.2 Quorum-based Repair Decisions 9.3 Implementation
- 9.3.1 DistributedRepairCoordinator Class
- 9.3.2 Voting Mechanisms
- 9.3.3 Partition Tolerance 9.4 Availability Modeling
- 9.4.1 $A_{\text{distributed}} = 1 - (1 - A_{\text{node}})^N$

Gap 10: Formal Specification Languages

10.1 Mathematical Framework

- 10.1.1 Linear Temporal Logic (LTL)
- 10.1.2 Hoare Logic $\{P\}C\{Q\}$
- 10.1.3 Signal Temporal Logic (STL) 10.2 Safety Property Encoding
- 10.2.1 Stability Invariants: $\Box(p(A) < 1)$
- 10.2.2 Convergence: $\Diamond(\text{loss} < \epsilon)$
- 10.2.3 Monotonicity Constraints 10.3 Implementation
- 10.3.1 FormalSpecificationChecker Class
- 10.3.2 Z3 SMT Solver Integration
- 10.3.3 Property Verification 10.4 Runtime Monitoring
- 10.4.1 LTL Trace Checking

Gap 11: Quantitative Interpretability Metrics

11.1 Mathematical Framework

- 11.1.1 Complexity Score IC
- 11.1.2 Fidelity Score F
- 11.1.3 Consistency Score C
- 11.1.4 Attribution Stability AS 11.2 Rule Analysis
- 11.2.1 Contradiction Detection
- 11.2.2 Redundancy Identification 11.3 Implementation
- 11.3.1 InterpretabilityMetrics Class
- 11.3.2 Logic Complexity Computation
- 11.3.3 Fidelity Measurement 11.4 CondorNet Predicate Analysis
- 11.4.1 8-Predicate Complexity
- 11.4.2 Set/SuperSet Metrics

Gap 12: Neuroscience-Inspired Computational Models

12.1 Mathematical Framework

- 12.1.1 Homeostatic Plasticity: $\Delta w \propto (r_{\text{target}} - r_{\text{actual}})$

- 12.1.2 Synaptic Pruning Rules
 - 12.1.3 STDP: $\Delta w = f(\Delta t_{\text{spike}})$
 - 12.2 Bio-inspired Algorithms
 - 12.2.1 Homeostatic Weight Adjustment
 - 12.2.2 Activity-dependent Pruning
 - 12.3 Implementation
 - 12.3.1 NeuroscienceInspiredRepair Class
 - 12.3.2 Firing Rate Regulation
 - 12.3.3 Temporal Credit Assignment
 - 12.4 Integration with CondorNet
 - 12.4.1 Predicate Gate Plasticity
-

1.1 Mathematical foundation for Lyapunov stability in CondorNet ETD-1

1.1.1 Lyapunov function construction for the ETD-1 CondorNet master equation

We start from the CondorNet ETD-1 master equation in continuous time, written at the level of the augmented state:

$$\frac{dx}{dt} = A_{\theta}(u)x + B_{\theta}(u) + G_{\theta}(x, u)\Delta X + D(\text{Greeks}, r_{k-1}, q)$$

where:

- **State vector:**

$$x \in \mathbb{R}^{d_x}, x = \begin{bmatrix} h \\ v \\ m \\ r \end{bmatrix}$$

with block dimensions

$$d_x = d_h + d_v + d_m + d_r$$

- **Linear dynamics:** $A_{\theta}(u) \in \mathbb{R}^{d_x \times d_x}$ is the learned “A-matrix” (possibly modulated by control/inputs u).
- **Control/forcing:** $B_{\theta}(u)$ encodes control-like terms; $G_{\theta}(x, u)\Delta X$ and $D(\cdot)$ encode stochastic and exogenous market forcing (Greeks, rates, dividends, etc.).

For **stability analysis**, we first strip away stochastic and exogenous forcing to isolate the core linear dynamics:

$$\frac{dx}{dt} = A_{\theta}x$$

This is the canonical linear time-invariant (LTI) system we will certify as stable.

We choose a **quadratic Lyapunov candidate**:

$$V(x) = x^\top P x$$

with:

- $x \in \mathbb{R}^{d_x}$ the augmented CondorNet state,
- $P \in \mathbb{R}^{d_x \times d_x}$ a **symmetric positive-definite** matrix:

$$P = P^\top, P \succ 0$$

This is the standard Lyapunov form used in control theory because:

- It is **globally defined** on $\mathbb{R}^{d_x}$,
- It induces a norm $\|x\|_P^2 = x^\top P x$,
- It yields a tractable derivative along trajectories.

To qualify as a Lyapunov function, V must satisfy:

1. **Positive definiteness** (energy-like):

$$\exists \alpha_1, \alpha_2 > 0 \text{ such that } \alpha_1 \|x\|^2 \leq V(x) \leq \alpha_2 \|x\|^2, \forall x$$

where $\|\cdot\|$ is the Euclidean norm. For quadratic forms:

- $\alpha_1 = \lambda_{\min}(P)$,
- $\alpha_2 = \lambda_{\max}(P)$.

2. **Negative definiteness of the derivative** along system trajectories:

$$\frac{dV}{dt}(x) \leq -\alpha_3 \|x\|^2, \alpha_3 > 0$$

Compute the derivative:

$$\begin{aligned} \frac{dV}{dt} &= \frac{d}{dt}(x^\top P x) = \dot{x}^\top P x + x^\top P \dot{x} \\ &= (A_\theta x)^\top P x + x^\top P (A_\theta x) \\ &= x^\top A_\theta^\top P x + x^\top P A_\theta x \\ &= x^\top (A_\theta^\top P + P A_\theta) x \end{aligned}$$

We want:

$$\frac{dV}{dt} = x^\top (A_\theta^\top P + P A_\theta) x = -x^\top Q x$$

with:

- $Q \in \mathbb{R}^{d_x \times d_x}$ symmetric positive-definite:

$$Q = Q^\top, Q \succ 0$$

Thus the **Lyapunov equation** is:

$$A_\theta^\top P + P A_\theta = -Q$$

If there exists $P \succ 0$ solving this for a given $Q \succ 0$, then:

$$\frac{dV}{dt} = -x^\top Q x \leq -\lambda_{\min}(Q) \|x\|^2$$

and the system is **exponentially stable**.

CondorNet interpretation:

- A_θ is the learned ETD-1 A-matrix over the 4-block state $[h; v; m; r]$.
- P is a **global energy metric** over the entire CondorNet state space.
- Q encodes how aggressively we want energy to decay (larger $Q \rightarrow$ faster decay).

This gives us a **rigorous, global stability certificate** for the CondorNet dynamics.

1.1.2 Continuous Algebraic Riccati Equation (CARE) and Lyapunov equation

For systems with explicit control inputs, the more general **Continuous Algebraic Riccati Equation (CARE)** appears:

$$A^\top P + P A - P B R^{-1} B^\top P + Q = 0$$

where:

- $A \in \mathbb{R}^{d_x \times d_x}$ is the system matrix,
- $B \in \mathbb{R}^{d_x \times d_u}$ is the input matrix,
- $R \in \mathbb{R}^{d_u \times d_u}$ is a positive-definite control cost matrix,
- $Q \in \mathbb{R}^{d_x \times d_x}$ is a state cost matrix.

In our **pure stability** setting (no explicit control law), we use the **special case**:

$$B = 0 \Rightarrow A^\top P + P A + Q = 0$$

This is the **continuous-time Lyapunov equation**.

For CondorNet, we typically choose:

- $Q = \alpha I$ with $\alpha > 0$ to keep the problem well-conditioned,
- Solve for P given a fixed A_θ .

Numerical solution (dense case):

Using SciPy:

```
1 from scipy.linalg import solve_continuous_lyapunov
2
3 # A: (d_x, d_x) numpy array
4 # Q: (d_x, d_x) positive-definite numpy array
5 P = solve_continuous_lyapunov(A.T, Q)
```

This solves:

$$A^T P + P A + Q = 0$$

for P . If A is Hurwitz (all eigenvalues with negative real part), a unique $P \succ 0$ exists for any $Q \succ 0$.

Computational complexity:

- Dense solver: $O(d_x^3)$.
- For CondorNet with $d_x \approx 192$, this is entirely tractable offline and even periodically online.

For **very large** systems, one can use:

- Krylov subspace methods,
- Low-rank approximations,
- Iterative Lyapunov solvers.

But for CondorNet's current dimensionality, the standard dense solver is sufficient.

1.1.3 Exponential stability proof and decay rate

Assume we have found $P \succ 0$ and $Q \succ 0$ such that:

$$A^T P + P A = -Q$$

Define:

$$V(x) = x^{\top} P x$$

Then:

$$\frac{dV}{dt} = -x^{\top} Q x$$

Using eigenvalue bounds:

- Since $P \succ 0$:

$$\lambda_{\min}(P) \|x\|^2 \leq V(x) \leq \lambda_{\max}(P) \|x\|^2$$

- Since $Q \succ 0$:

$$x^{\top} Q x \geq \lambda_{\min}(Q) \|x\|^2$$

Thus:

$$\frac{dV}{dt} = -x^{\top} Q x \leq -\lambda_{\min}(Q) \|x\|^2$$

and:

$$\|x\|^2 \geq \frac{1}{\lambda_{\max}(P)} V(x)$$

So:

$$\frac{dV}{dt} \leq -\frac{\lambda_{\min}(Q)}{\lambda_{\max}(P)} V(x)$$

Let:

$$c = \frac{\lambda_{\min}(Q)}{\lambda_{\max}(P)} > 0$$

Then:

$$\frac{dV}{dt} \leq -c V(x)$$

Integrating:

$$V(t) \leq V(0) e^{-ct}$$

Using the bounds between V and $\|x\|^2$:

$$\lambda_{\min}(P) \|x(t)\|^2 \leq V(t) \leq V(0) e^{-ct} \leq \lambda_{\max}(P) \|x(0)\|^2 e^{-ct}$$

So:

$$\|x(t)\|^2 \leq \frac{\lambda_{\max}(P)}{\lambda_{\min}(P)} \|x(0)\|^2 e^{-c}$$

Taking square roots:

$$\|x(t)\| \leq \sqrt{\frac{\lambda_{\max}(P)}{\lambda_{\min}(P)}} \|x(0)\| e^{-\lambda t}$$

with:

$$\lambda = \frac{c}{2} = \frac{\lambda_{\min}(Q)}{2 \lambda_{\max}(P)}$$

This is **global exponential stability** with rate λ .

CondorNet-specific numbers:

- Typical block sizes: $d_h = 128, d_v = 16, d_m = 32, d_r = 16 \rightarrow d_x \approx 192$.
- We can target a decay rate $\lambda > 0.01$, which implies a characteristic time constant of about $1/\lambda \approx 100$ time units (e.g., timesteps/epochs depending on scaling).
- This gives a **quantitative convergence guarantee** for the ETD-1 dynamics.

1.1.4 Region of attraction as an ellipsoid in CondorNet state space

Given:

$$V(x) = x^\top P x$$

and a level set:

$$\Omega_c = \{x \in \mathbb{R}^{d_x} : V(x) \leq c\}$$

we have:

$$x^\top P x \leq c$$

Since $P \succ 0$, this defines an **ellipsoid** in $\mathbb{R}^{d_x}$.

Let $\lambda_1, \dots, \lambda_{d_x}$ be the eigenvalues of P , and $P = U\Lambda U^\top$ with $\Lambda = \text{diag}(\lambda_i)$. In coordinates $y = U^\top x$:

$$x^\top P x = y^\top \Lambda y = \sum_{i=1}^{d_x} \lambda_i y_i^2 \leq c$$

So:

$$\sum_{i=1}^{d_x} \frac{y_i^2}{c/\lambda_i} \leq 1$$

The **semi-axis lengths** of the ellipsoid are:

$$a_i = \sqrt{\frac{c}{\lambda_i}}, i = 1, \dots, d_x$$

The **volume** of this ellipsoid is:

$$\text{Vol}(\Omega_c) = \frac{\pi^{d_x/2}}{\Gamma(d_x/2 + 1)} \prod_{i=1}^{d_x} a_i = \frac{\pi^{d_x/2}}{\Gamma(d_x/2 + 1)} \frac{c^{d_x/2}}{\sqrt{\det P}}$$

This gives a **quantitative region of attraction**:

- Larger $c \rightarrow$ larger ellipsoid,
- Better-conditioned P (smaller $\det P$) $\rightarrow$ larger volume for fixed c .

Maximum safe initial condition:

If we have a **safe region** $\mathcal{S} \subset \mathbb{R}^{d_x}$ (e.g., bounds on hidden states, volatility states, etc.), we can choose c such that:

$$\Omega_c \subset \mathcal{S}$$

and define:

$$c^* = \max \{c: \Omega_c \subset \mathcal{S}\}$$

Then any initial condition $x(0)$ with $V(x(0)) \leq c^*$ is guaranteed to remain in $\mathcal{S}$ and converge to the origin.

For CondorNet, this gives a **geometric safety envelope** in the 4-block state space.

1.2 Stability-preserving repair constraints

1.2.1 Negative definiteness verification for repaired A-matrices

Suppose we have a **baseline stable** matrix A_{old} with Lyapunov matrix P satisfying:

$$A_{\text{old}}^T P + P A_{\text{old}} = -Q, Q > 0$$

Now we apply a **surgical repair**:

$$A_{\text{new}} = A_{\text{old}} + \Delta A$$

We want to ensure that the repaired system remains stable. A sufficient condition is:

$$A_{\text{new}}^T P + P A_{\text{new}} < 0$$

i.e., the symmetric matrix:

$$M = A_{\text{new}}^T P + P A_{\text{new}}$$

is **negative definite** (all eigenvalues strictly negative).

Exact check (eigenvalues):

```
python
import numpy as np

def verify_repair_preserves_stability(A_new: np.ndarray, P: np.ndarray) ->
bool:
    """
    Check if A_new^T P + P A_new is negative definite.

    Method: compute eigenvalues; all must be < 0.
    """
    M = A_new.T @ P + P @ A_new
    eigenvalues = np.linalg.eigvalsh(M) # symmetric → eigvalsh

    return np.all(eigenvalues < 0)
```

- Complexity: $O(d_x^3)$.
- For $d_x \approx 192$, this is feasible as a **post-repair check**.

Fast conservative check (Gershgorin discs):

We can approximate the negative definiteness using the **Gershgorin Circle Theorem**:

- Each eigenvalue λ of M lies in at least one disc:

$$\lambda \in \{z \in \mathbb{C}: |z - M_{ii}| \leq R_i\}$$

where:

$$R_i = \sum_{j \neq i} |M_{ij}|$$

If for every row i :

$$M_{ii} + R_i < 0$$

then **all discs lie strictly in the left half-plane**, and thus all eigenvalues are negative.

```

1  def gershgorin_check(A_new: np.ndarray, P: np.ndarray) -> bool:
2      """
3          Approximate stability check using Gershgorin bounds.
4
5          Theorem:  $\lambda_i \in \text{Disc}(M_{ii}, \sum_{j \neq i} |M_{ij}|)$ .
6          If for all  $i$ , center + radius < 0, then all eigenvalues < 0.
7      """
8      M = A_new.T @ P + P @ A_new
9      d = M.shape[0]
10
11     for i in range(d):
12         center = M[i, i]
13         radius = np.sum(np.abs(M[i, :])) - np.abs(M[i, i])
14
15         # If disc crosses or touches non-negative real axis, we cannot
16         guarantee negativity
17         if center + radius >= 0:
18             return False
19
20     return True

```

- Complexity: $O(d_x^2)$.
- This is ideal for **fast screening** of candidate repairs before doing a full eigenvalue check.

1.2.2 Spectral bounds from the Lyapunov matrix

The Lyapunov equation also yields **bounds on the spectral properties** of A .

If:

$$A^T P + P A = -Q, P > 0, Q > 0$$

then one can derive bounds on the **spectral norm** $\sigma_{\max}(A)$ and the **spectral radius** $\rho(A)$.

A useful inequality (one of several possible forms) is:

$$\sigma_{\max}(A) \leq \frac{\lambda_{\max}(P^{-1}Q)}{2 \lambda_{\min}(P)}$$

and, more fundamentally:

$$\rho(A) < 0 (\text{continuous-time stability}) \Leftrightarrow \exists P \succ 0, Q \succ 0: A^T P + P A = -Q$$

For **discrete-time** stability (e.g., ETD-1 step operator), we want:

$$\rho(A_{\text{disc}}) < 1$$

and can relate the continuous-time A to the discrete-time operator via the ETD-1 scheme. In practice, we use:

- **Spectral radius check** on the learned A -matrix,
- **Lyapunov feasibility** as a stronger, global certificate.

Choosing $Q = I$ (identity) often yields a well-conditioned P and simplifies interpretation.

1.3 Implementation

1.3.1 LyapunovStabilityMonitor class

We now package the above mathematics into a reusable, real-time monitoring object that can be attached to CondorNet.

```

1 import numpy as np
2 from scipy.linalg import solve_continuous_lyapunov, eigvalsh
3 from scipy.special import gamma
4
5 class LyapunovStabilityMonitor:
6     """
7     Real-time Lyapunov stability monitoring for CondorNet ETD-1.
8
9     Mathematical foundation:
10     -  $V(x) = x^T P x$  (Lyapunov function)
11     -  $dV/dt = x^T (A^T P + P A) x$  (derivative along trajectories)
12     - Stability:  $A^T P + P A = -Q$  with  $P, Q \succ 0$ 
13
14     Attributes:
15     d_x: State dimension (d_h + d_v + d_m + d_r)
16     P: Lyapunov matrix (d_x × d_x)
17     Q: Weight matrix for Lyapunov equation
18     decay_rate: Exponential decay rate  $\lambda$ 
19     alpha_1, alpha_2: bounds such that
20         alpha_1 ||x||^2 ≤ V(x) ≤ alpha_2 ||x||^2
21     """
22 
```

```

23 def __init__(self, d_x: int, Q_weight: float = 1.0):
24     """
25     Initialize Lyapunov monitor.
26
27     Args:
28         d_x: State dimension
29         Q_weight: Weight for Q matrix (Q = Q_weight * I)
30     """
31     self.d_x = d_x
32     self.Q = Q_weight * np.eye(d_x)
33     self.P = None
34     self.decay_rate = None
35     self.alpha_1 = None # lower bound on V(x)/||x||^2
36     self.alpha_2 = None # upper bound on V(x)/||x||^2
37
38 def solve_lyapunov_equation(self, A: np.ndarray) -> bool:
39     """
40     Solve A^T P + P A + Q = 0 for P.
41
42     Args:
43         A: System matrix (d_x x d_x)
44
45     Returns:
46         True if solution found and P is positive definite, False
47 otherwise.
48     """
49     try:
50         # Solve continuous-time Lyapunov equation
51         self.P = solve_continuous_lyapunov(A.T, self.Q)
52
53         # Check P > 0 via eigenvalues
54         eigenvalues_P = eigvalsh(self.P)
55         if np.min(eigenvalues_P) <= 0:
56             print(f"[LYAPUNOV WARNING] P not positive definite: "
57                   f"λ_min={np.min(eigenvalues_P):.6f}")
58             return False
59
60         # Bounds α1, α2 from eigenvalues of P
61         self.alpha_1 = float(np.min(eigenvalues_P))
62         self.alpha_2 = float(np.max(eigenvalues_P))
63
64         # Decay rate λ = λmin(Q) / (2 λmax(P))
65         lambda_min_Q = float(np.min(eigvalsh(self.Q)))
66         self.decay_rate = lambda_min_Q / (2.0 * self.alpha_2)
67
68         return True
69
70     except np.linalg.LinAlgError as e:
71         print(f"[LYAPUNOV ERROR] Failed to solve Lyapunov equation: "
72               f"{e}")
73         return False
74

```



```

75 def compute_lyapunov_value(self, x: np.ndarray) -> float:
76     """
77     Compute  $V(x) = x^T P x$ .
78
79     Args:
80         x: State vector ( $d_x$ ,)
81
82     Returns:
83         Scalar Lyapunov value  $V(x)$ .
84     """
85     if self.P is None:
86         raise ValueError("Lyapunov equation not solved. "
87                           "Call solve_lyapunov_equation() first.")
88
89     return float(x.T @ self.P @ x)
90
91 def compute_lyapunov_derivative(
92     self,
93     x: np.ndarray,
94     A: np.ndarray,
95     B_u: np.ndarray = None
96 ) -> float:
97     """
98     Compute  $dV/dt$  along trajectories.
99
100     For  $dx/dt = A x + B u$ :
101          $dV/dt = x^T (A^T P + P A) x + 2 x^T P B u$ 
102
103     Args:
104         x: State vector ( $d_x$ ,)
105         A: System matrix ( $d_x \times d_x$ )
106         B_u: Control/forcing term ( $d_x$ ,) or None
107
108     Returns:
109         Scalar derivative  $dV/dt$ .
110     """
111     if self.P is None:
112         raise ValueError("Lyapunov equation not solved.")
113
114     # Quadratic term
115     A_lyap = A.T @ self.P + self.P @ A
116     quadratic = float(x.T @ A_lyap @ x)
117
118     # Linear term (if control present)
119     linear = 0.0
120     if B_u is not None:
121         linear = 2.0 * float(x.T @ self.P @ B_u)
122
123     return quadratic + linear
124
125 def is_stable(
126     self,

```

```

127         x: np.ndarray,
128         A: np.ndarray,
129         B_u: np.ndarray = None,
130         threshold: float = 0.0
131     ) -> bool:
132         """
133         Check stability at state x: dV/dt < threshold (typically 0).
134
135         Args:
136             x: State vector
137             A: System matrix
138             B_u: Control/forcing term (optional)
139             threshold: Stability threshold (default 0)
140
141         Returns:
142             True if dV/dt < threshold, False otherwise.
143         """
144         dV_dt = self.compute_lyapunov_derivative(x, A, B_u)
145         return dV_dt < threshold
146
147     def region_of_attraction(self, c: float = 1.0) -> dict:
148         """
149         Compute ellipsoidal region of attraction:
150
151              $\Omega_c = \{ x : x^T P x \leq c \}$ 
152
153         Args:
154             c: Level set value
155
156         Returns:
157             dict with semi_axes, volume, eigenvalues of P.
158         """
159         if self.P is None:
160             raise ValueError("Lyapunov equation not solved.")
161
162         eigenvalues_P = eigvalsh(self.P)
163         semi_axes = np.sqrt(c / eigenvalues_P)
164
165         # Volume = ( $\pi^{(d/2)} / \Gamma(d/2 + 1)$ ) ·  $c^{(d/2)} / \sqrt{\det(P)}$ 
166         d = self.d_x
167         volume_coeff = (np.pi ** (d / 2.0)) / gamma(d / 2.0 + 1.0)
168         volume = volume_coeff * (c ** (d / 2.0)) /
169 np.sqrt(np.linalg.det(self.P))
170
171         return {
172             "c": c,
173             "semi_axes": semi_axes.tolist(),
174             "volume": float(volume),
175             "eigenvalues_P": eigenvalues_P.tolist(),
176         }
177
178     def verify_repair(self, A_new: np.ndarray) -> bool:

```

```

179         """
180         Verify that a repaired A_new preserves stability:
181
182          $A_{\text{new}}^T P + P A_{\text{new}} < 0$  (negative definite)
183
184         Args:
185             A_new: Repaired system matrix
186
187         Returns:
188             True if all eigenvalues of  $A_{\text{new}}^T P + P A_{\text{new}}$  are negative.
189         """
190         if self.P is None:
191             raise ValueError("Lyapunov equation not solved.")
192
193         M = A_new.T @ self.P + self.P @ A_new
194         eigenvalues_M = eigvalsh(M)
195
196         is_stable = np.all(eigenvalues_M < 0)
197         if not is_stable:
198             max_eig = float(np.max(eigenvalues_M))
199             print(f"[REPAIR WARNING] Stability violated: "
200                   f"max( $\lambda(A^T P + P A)$ ) = {max_eig:.6f}")
201
202         return is_stable
203
204     def gershgorin_stability_check(self, A_new: np.ndarray) -> bool:
205         """
206         Fast approximate stability check using Gershgorin Circle Theorem.
207
208         Faster than full eigenvalue computation:  $O(d^2)$  vs  $O(d^3)$ .
209
210         Args:
211             A_new: Candidate repaired matrix
212
213         Returns:
214             True if all Gershgorin discs lie strictly in the left half-
215 plane.
216         """
217         if self.P is None:
218             raise ValueError("Lyapunov equation not solved.")
219
220         M = A_new.T @ self.P + self.P @ A_new
221         d = M.shape[0]
222
223         for i in range(d):
224             center = M[i, i]
225             radius = np.sum(np.abs(M[i, :])) - np.abs(M[i, i])
226
227             # Disc: [center - radius, center + radius]
228             if center + radius >= 0:
229                 return False
230

```

```

231         return True
232
233     def predict_trajectory(self, x0: np.ndarray, A: np.ndarray, T: int =
234 100) -> np.ndarray:
235         """
236         Predict state trajectory under exponential stability:
237
238              $x(t) \approx x(0) \cdot e^{-\lambda t}$ 
239
240         This is a coarse approximation using the decay rate  $\lambda$ .
241
242         Args:
243             x0: Initial state (d_x,)
244             A: System matrix (unused in this simple approximation)
245             T: Number of timesteps
246
247         Returns:
248             trajectory: (T, d_x) array of predicted states.
249         """
250         if self.decay_rate is None:
251             raise ValueError("Decay rate not computed. "
252                               "Call solve_lyapunov_equation() first.")
253
254         trajectory = np.zeros((T, self.d_x))
255         x = x0.copy()
256
257         for t in range(T):
258             trajectory[t] = x
259             x = x * np.exp(-self.decay_rate)
260
261         return trajectory
262
263     def get_metrics(self) -> dict:
264         """
265         Get comprehensive Lyapunov metrics.
266
267         Returns:
268             dict with P condition number, bounds, decay rate, etc.
269         """
270         if self.P is None:
271             return {"status": "not_initialized"}
272
273         return {
274             "status": "initialized",
275             "P_condition_number": float(np.linalg.cond(self.P)),
276             "alpha_1": float(self.alpha_1),
277             "alpha_2": float(self.alpha_2),
278             "decay_rate_lambda": float(self.decay_rate),
279             "convergence_time_95pct": (
280                 float(-np.log(0.05) / self.decay_rate)
281                 if self.decay_rate and self.decay_rate > 0
282                 else float("inf")

```

```
    ),  
    "P_eigenvalues_min_max": [float(self.alpha_1),  
float(self.alpha_2)],  
    "Q_trace": float(np.trace(self.Q)),  
}
```

